Universidad Nacional Autónoma de México
Facultad de Ingeniería

Computación gráfica e interacción humano-computadora

Semestre 2027-1

Manual de Usuario

Profesor: Eden Espinoza Urzua

Alumnos:
Vilchis Testas Alan Gael
Sandoval Gonzalez Brandon

Números de cuenta:
331634421
321009298

Grupo : 04

Link del repositorio:
https://github.com/alanvilchisssss/Blender_activities

Fecha de entrega: 09-11-2026

Introducción

Este documento describe el diseño y funcionamiento del proyecto; una recreación 3D interactiva de una calle urbana con edificios, una glorieta y vehículos , adicional con un sistema de metro. El proyecto está desarrollado en C + + con OpenGl 3.3 sobre el código proporcionado por el profesor, el entorno se modeló en blender y los objetos se exportaron en OBJ para generar sus animaciones por medio de código.

Objetivos:
Aplicar los conocimientos del curso de Computación Gráfica para recrear una escena en 3D por medio de OpenGL de una calle con ambientacion, iluminacion y animaciones coherentes; representar la escena con la mayor fidelidad posible, cargar la escena usando como base el código proporcionado por el profesor y asentar los conocimientos sobre el manejo de fuentes de luz y la creación de objetos.

Alcance:
La creación de una calle con edificios, banquetas, un metro, una glorieta con monumento, semáforos y letreros, además de implementar una cámara libre con iluminación dinámica y un ciclo de dia y noche con cinco animaciones, animando el metro, los coches y sus llantas para poder representar de mejor manera el comportamiento correcto de los vehículos.

Limitantes:
- No existen colisiones entre los vehículos que modelamos, los coches avanzan en fila y jamás se tocan
- El cielo es un color degradado por la hora del dia, no se usa el HDRI de blender
- Los materiales se aproximan con color base, textura, brillo y transparencia
- Las animaciones puestas en blender no se importan, para generarlas por código
- La carga inicial puede tardar por que se leen bastantes modelos OBJ
- El ejecutable necesita las carpetas models/ y shader/ junto a él

Metodología de software:
El proyecto se dividió en incrementos pequeños al final de cada uno existe una versión ejecutable que se probaba y se refinaban. Este modelo se eligió porque el alcance se fue precisando conforme se veía el resultado en la pantalla y además permite detectar errores en momentos tempranos del desarrollo, en el repositorio de github se pueden observar los commits realizados con mensajes descriptivos que nos permiten entender los cambios que realizamos.

Diagrama de Flujo:

El software funciona en 2 etapas. La primera ocurre una sola vez en blender y prepara los datos; la segunda es el programa en C + + que se ejecuta.

Etapa 1:
- Se recorre la escena y se descartan los objetos ocultos y widgets del rig
- Cada geometría distinta se escribe una vez como OBJ en su espacio local

- Se guarda un layout como la matriz del mundo

Etapa 2:

- Se inicializan GLFW y GLEW y se compila el shader
- Se carga el layout, se leen los modelos con Assimp y se clasifican los objetos por nombre y jerarquía
- Se prepara la ruta y el estado inicial de los coches
- Mientras la ventana siga abierta, el programa lee la entrada, actualiza las animaciones, calcula la iluminación para simular un ciclo de dia y noche
- Al cerrar la ventana se liberar los recursos

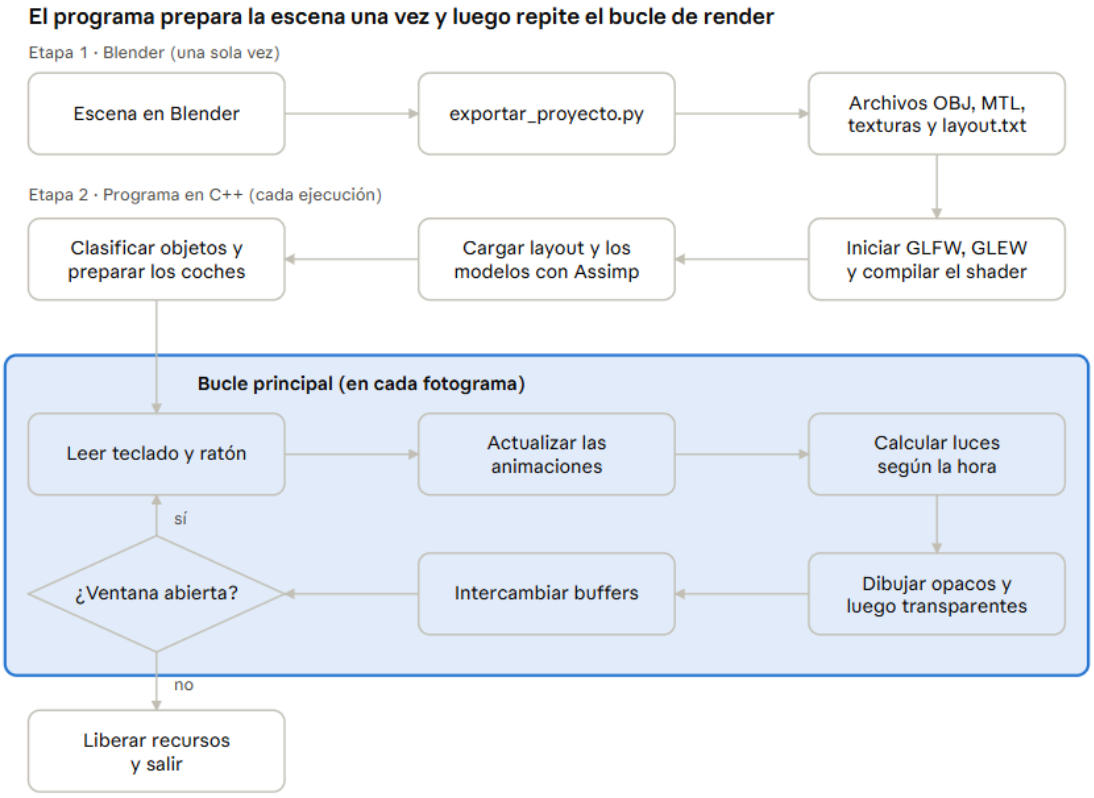


Diagrama de Gantt:

Actividad	S1	S2	S3	S4	S5	S6	S7	S8
Analisis y planeacion								
Modelado y ambientación								
Exportación								
Integración en OpenGL								
Animaciones 1								
Animaciones 2								

Pruebas y optimización								
Documentación								
Video y presentación								

Documentación:

Archivo	**Origen**	**Uso**
Main.cpp	Nuevo	Carga la escena y el bucle de render
Ruta.h	Nuevo	Ruta cerrada de los coches
Model.h	Base	Carga de OBJ con Assimp
Mesh.h	Base	Buffers de OpenGL
Shader.h	Base	Compilación y enlace de shader
Camera.h	Base	Cámara libre
Shader/Lighting.vs	Base	Transforma vértices
Shader/Lighting.frag	Base	Iluminación de Phong

Estructura de datos y funciones del código principal "main.cpp":

Funcion	**Uso**
CargaEscena(ruta)	Carga cada OBJ una sola vez y calcula el centro y radio de cada objeto
ClasificarObjetos()	Asigna categoria y estilo por nombre y raíz
RutaCoches::construit()	Calcula acumulados, rumbo y curva de los coches
PrepararCoches()	Fija el punto de referencia de cada coche
ActualizarCoches(dt)	Avanza cada coche según el deltatime para que no existan saltos en el movimiento
EstadoMetro(t,...)	Devuelve el desplazamiento del tren para el uso de la máquina de estados
Easeout, easeIn, smooth	Funciones de suavizado para el movimiento

	de las puertas del metro
DoMovement	Mueve la cámara
KeyCallback, MouseCallback	Entrada de teclado

Manual de usuario

1.- Cómo Iniciar:

Dentro de la carpeta de entrega de laboratorio se encuentra el ejecutable llamado ".exe", haz doble click sobre este, es necesario que el ejecutable no se mueva de donde es entregado pues necesita de la carpeta con los modelos y shaders para funcionar.

Puede tardar un poco, el proyecto tiene un par de modelos que necesita cargarse y esto puede llevar un pequeño lapso de tiempo.

También si se requiere lanzar el proyecto desde el código, podremos hacerlo sin preocuparnos de ejecutar o cambiar todas las rutas, a partir de un repositorio de github, con el que podremos descargar las bibliotecas y añadirlas a nuestro visual studio code, por medio de los siguientes comandos:

git clone https://github.com/microsoft/vcpkg.git cd vcpkg
.\bootstrap-vcpkg.bat
.\vcpkg.exe install glfw3 glad glm glew stb assimp soil soil2
.\vcpkg.exe integrate install

Dentro de nuestro repositorio hay un manual más extenso de como instalar estas bibliotecas para una mayor comodidad de nuestro código libre.

2.-Controles:

Control	**Acción**
W	Avanzar hacia donde miras
S	Retroceder
A	Moverse a la izquierda
D	Moverse a la derecha
Shift(Mantener)	Moverte más rápido
Ratón	Girar la vista de la cámara
N	Pausar o reanudar el ciclo de dia y noche
F	Encender o apagar la linterna
Esc	Cerrar el programa

Conclusiones:

En conclusión, el modelado y animación de situaciones o formas de ver nuevas escenas y momentos, por medio de la programación e instancias de objetos es una parte importante del desarrollo de la computación gráfica, pues a partir de ello es que podemos tener las vistas de nuestras computadoras, a partir de los simples conceptos del vértice, vector, caras y geometría de un modelo poder representar nuevas simulaciones con las cuales, ya sea científica o recreativa, tener una forma en la cuál podremos desarrollar aproximaciones a la realidad por medio de interpretaciones que realizamos primero nosotros, y luego que podremos pasar por medio de software CAD, esto de cierta forma facilita a la visualización de algo que quizá tenemos en mente, o simplemente es asimilado a la vida real, pero que no sucede en sí en la vida real. Es por ello que los fundamentos de entender la computación gráfica para nuestra ingeniería es fundamental por las aplicaciones a las que podemos dar forma, donde ya englobamos el entendimiento de cómo podemos pasar un modelo físico a algo digital, y seguir una cierta metodología de diseño para poder estandarizar y no simplemente resolverlo sin ningún fundamento u orden detrás. Aterrizando al proyecto, se aplicaron diferentes conceptos básicos de la computación gráfica, como lo fue el pipeline de renderizado, que al final del día no se maneja manualmente, pero que se debe de comprender desde el punto de vista computarizado para saber qué es lo que se realiza en la computadora, y control de fallos, así como la animación de objetos modelados por medio de blender, un motor visual de renderizado y construcción para poder tener una representación de algo de la vida real, que en este caso fue la escena ambientada en la Ciudad de México, los automóviles en la persecución, etc.